



Domain 1: Design Solutions for Organizational Complexity



Overview

The first domain of the AWS Certified Solutions Architect Professional exam evaluates your capability to implement solutions that allow different accounts and business units to operate in an AWS environment securely and reliably. As you become part of a larger organization, the number of users and stakeholders involved in your cloud architecture becomes more complex. You need to be able to segregate these user groups and business units according to their respective purpose and simplify their responsibilities within your AWS environments. Consequently, you also need to make sure that each group is given access to what they should and what they only need. This is to avoid any unnecessary access that could result in security leaks for the organization.

26% of questions in the actual SAP-C02 exam revolve around designing an organizational setup that involves the use of multiple AWS accounts, AWS Regions, VPCs, and billing configuration. This is the second biggest domain in the AWS Certified Solutions Architect Professional exam so expect to see a lot of questions that involve a variety of management and governance services in AWS. You have to focus on the following AWS services: AWS Organizations, AWS Control Tower, AWS Direct Connect, Amazon Route 53, AWS Security Hub, and others. Make sure that you know how Consolidated Billing works for multiple AWS accounts that are under an AWS Organization, especially the right configuration of enabling or disabling the Reserved Instance (RI) Sharing option for one or more AWS accounts.

This domain checks your know-how in doing these tasks:

- Architecting network connectivity strategies
- Prescribing security controls
- Designing reliable and resilient architectures.
- Designing a multi-account AWS environment
- Determining cost optimization and visibility strategies.

In this chapter, we will cover the related topics for organizational designs and strategies in AWS that will likely show up in your Solutions Architect Professional exam.

Managing of Multiple AWS Accounts in an Organization

As a company grows larger and the number of AWS users and resources increase, it becomes extraordinarily difficult to manage such a huge, complex ecosystem in just a single AWS account. Various teams will have different workloads, different stakeholders will have different objectives, and different environments will have different priorities. And much like in a software development lifecycle wherein you have a dedicated environment for development, for QA or staging, for UAT, and for production, AWS allows you to set up a similar structure at no cost through account organizations.

In an ideal scenario, you should be using one account per development lifecycle environment. You should also have a separate account for centrally storing logs, another separate account for facilitating security between the different accounts under your organization, and a separate account for billing and administration tasks. This is known as a **Landing Zone** setup.

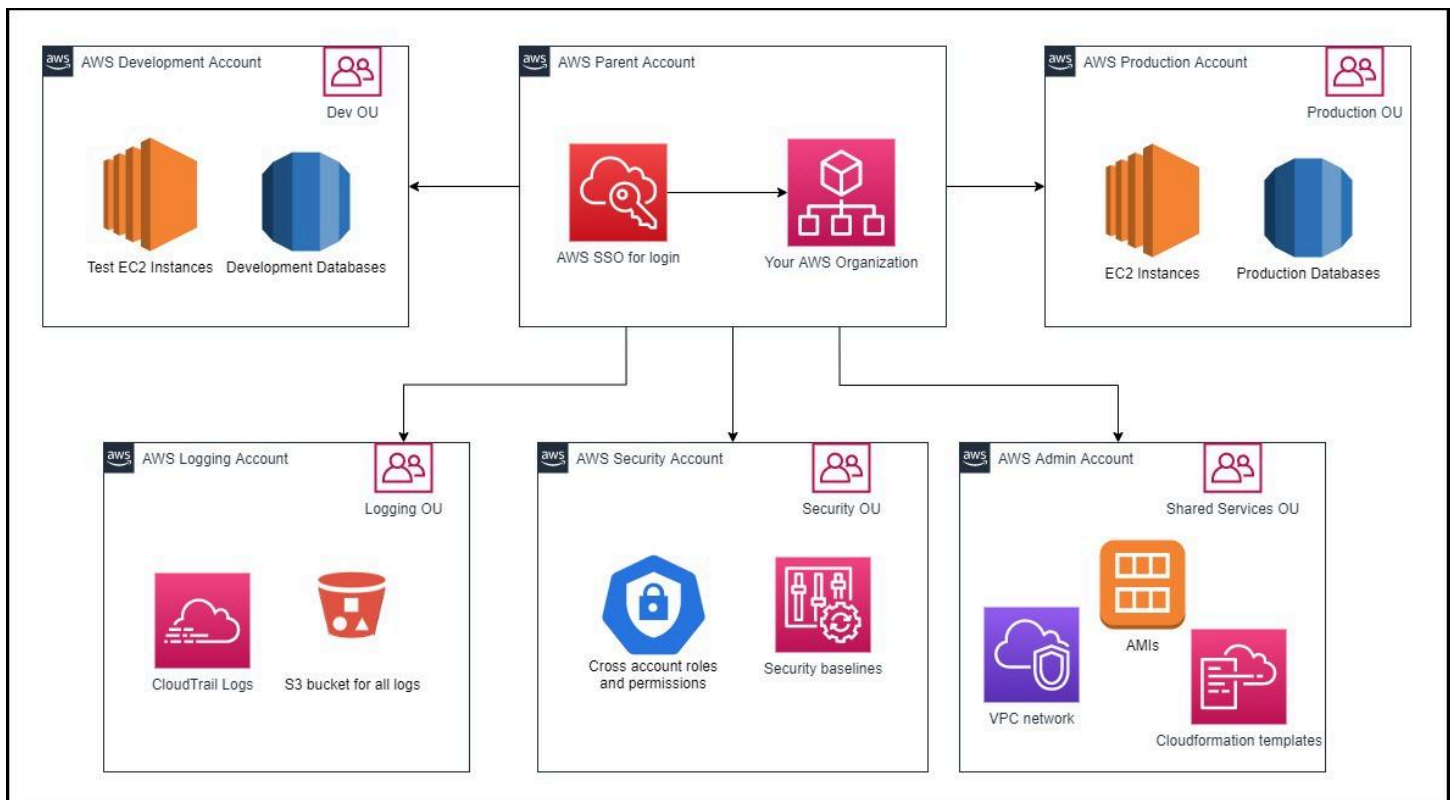


Figure: Example Setup of Landing Zone

Through this structure, you can experiment and develop faster since you have achieved a degree of isolation and flexibility. You can create an exact copy of an environment setup of another account and not have to worry about affecting the other account's processes while you define your own.

There are many ways to manage multiple accounts in AWS, but the most common and simplest method is by using AWS Organizations. This service allows you to govern and centrally manage your different AWS accounts under one account. It also provides many features for implementing security, cost management, and infrastructure compliance which we will also discuss along the way. The main components in AWS Organizations are the **master account** and the **member accounts**. As the name implies, the master account is where you'll be creating your organization. You can then invite other accounts to join your organization as members and manage them from your end. Take note that a member account can be a part of only one AWS Organization at a time. Once you have all your necessary accounts joined to the organization, you can start grouping them together into **Organization Units (OUs)**, which will allow you to create a hierarchy. Making use of OUs will not only simplify account management, but also enable you to easily deploy security policies and shared resources across multiple accounts in an OU at the same time.

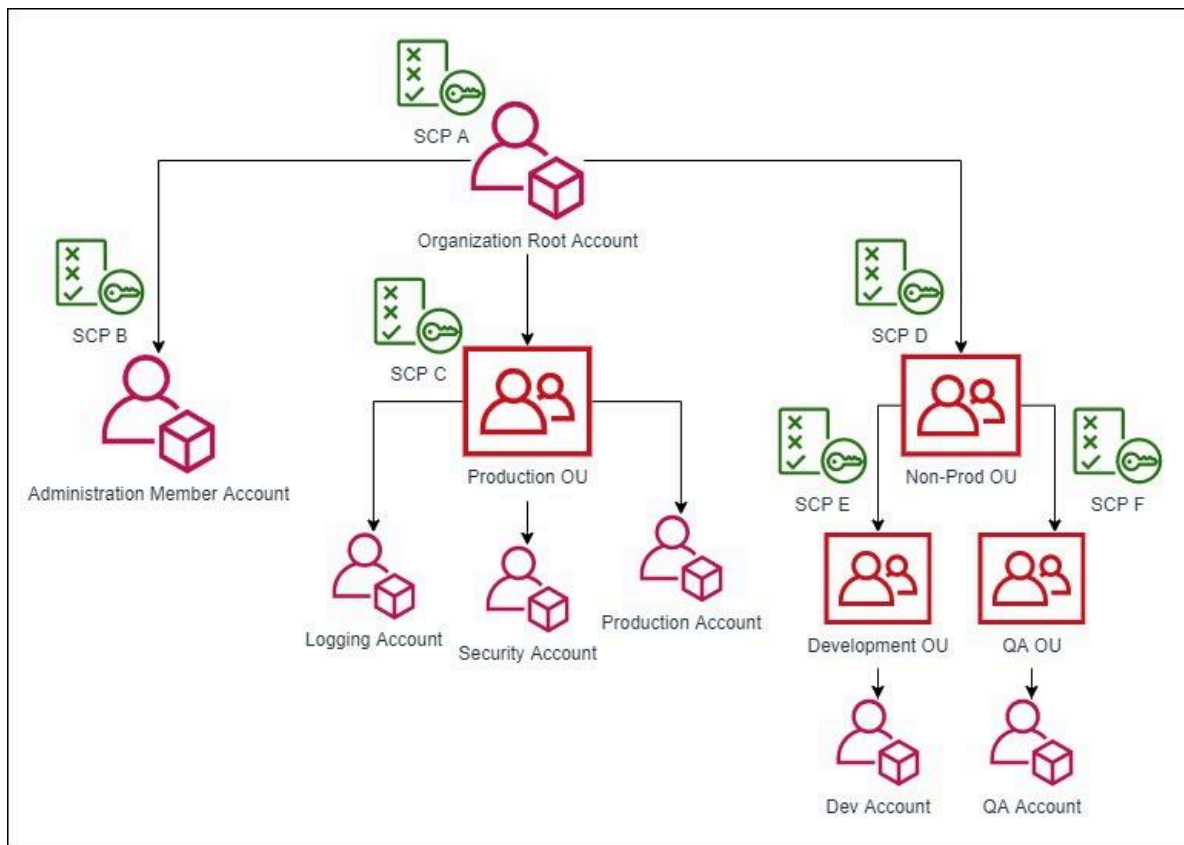


Figure: Example Structure of an AWS Organization with Multiple Accounts, OUs and SCPs

References:

<https://aws.amazon.com/solutions/implementations/aws-landing-zone/>
<https://aws.amazon.com/blogs/mt/tag/aws-multi-account-management/>
<https://aws.amazon.com/organizations/>

Security and Access Controls for a Multi-Account Structure

Since there can be hundreds of users and services interacting with one another in a multi-account structure, configuring security properly is vital in ensuring that you adhere to the principle of least privilege. There are different strategies that you can implement for multi-account security, depending on your business needs. There are also a few best practices that we will be discussing while you leverage these strategies. Sometimes, there can be questions in your exam that utilize more than one strategy for implementing security. The best way to know which to choose is to determine the assets involved in the accounts.

Cross-account roles

In a standalone account, IAM roles are a great way to provide access to your resources without having to create dedicated user credentials. They can also be attached to AWS services to allow interaction with one another in a secure manner. But what you might not have known is that you can also use IAM roles to provide access to users in another account. These are known as **cross-account roles**. Cross-account roles save you from the tedious task of creating and managing dedicated IAM Users in each account.

To get started with cross-account roles, you need to go to the IAM service and create a role meant for cross-account access. For convenience, imagine that you administer account A and the one requiring access to your environment is account B. During role creation, you input the Account ID of account B. At this point, you can also require users of account B to be MFA authenticated before they can assume the role.

Create role

1 2 3 4

Select type of trusted entity

 AWS service EC2, Lambda and others	 Another AWS account Belonging to you or 3rd party	 Web identity Cognito or any OpenID provider	 SAML 2.0 federation Your corporate directory
--	---	---	--

Allows entities in other accounts to perform actions in this account. [Learn more](#)

Specify accounts that can use this role

Account ID*

- Options**
- ☐ Require external ID (Best practice when a third party will assume this role)
 - ☐ Require MFA 

Figure: Create a cross-account role in IAM

On the final step, you provide the role with the necessary permissions to your account A via IAM Policies. Once this cross-account role has been created, IAM Users in account B can switch to or assume this role and gain

the permissions to do what they need to do in your account. To limit who can assume this role in account B, the admin of account B can create a policy that allows only specific users to assume the cross-account role.

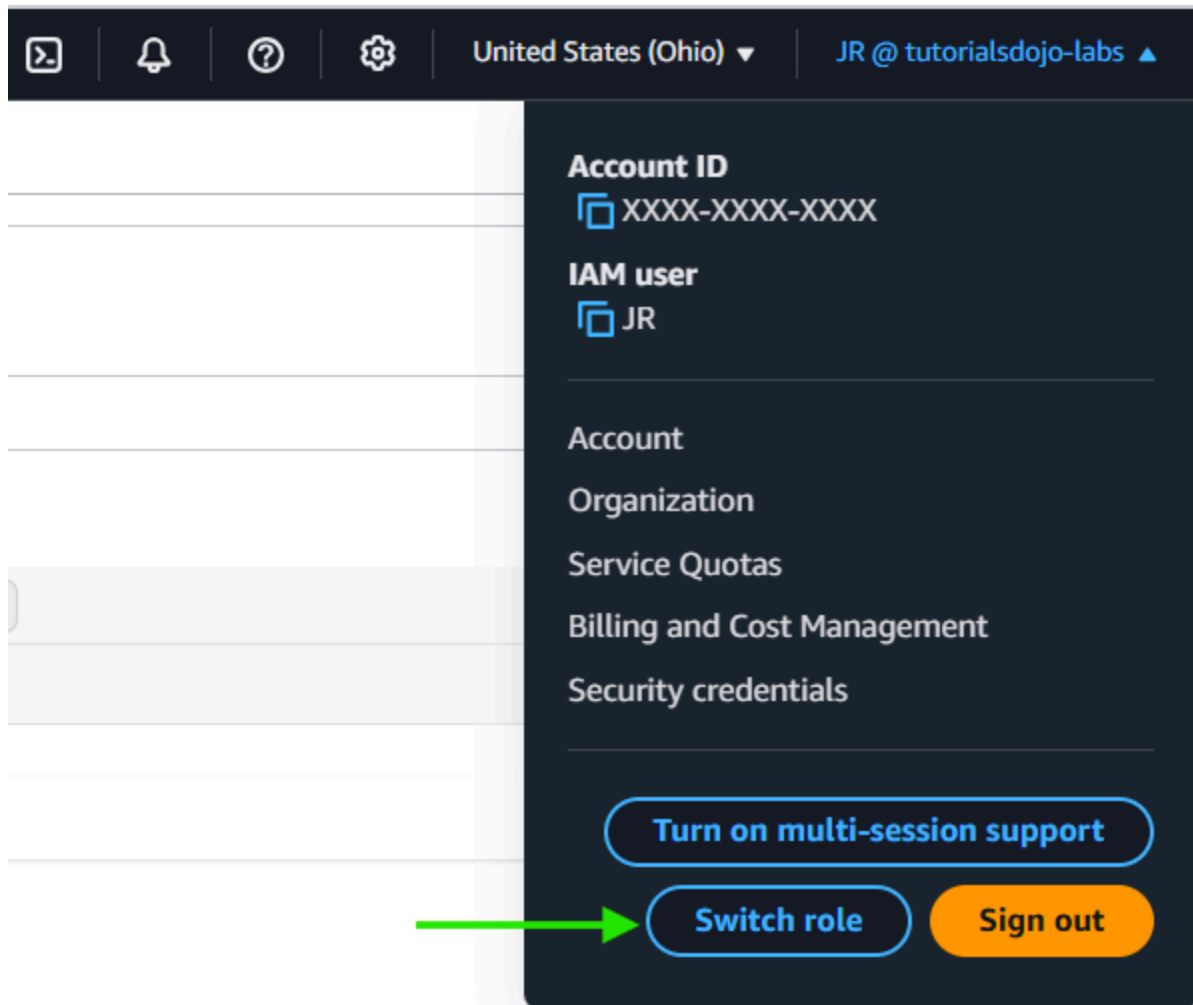


Figure: How to switch role in the AWS Console

AWS Organizations Service Control Policies

When you are the administrator of multiple accounts in an AWS Organization, you need to make sure that each account will function the way they are intended to. You can use Service Control Policies (SCPs) to restrict the actions that entities can perform in an account. SCPs are written in similar syntax as IAM Policies. SCPs apply account-wide, so it affects both IAM Users and IAM Roles managed by that account, but it does not affect resource-based policies/service-linked roles. Do keep in mind that you can only use SCPs if you have enabled **all features** in your AWS Organization.



SCPs can be attached to individual accounts and OUs. Attaching an SCP to an OU cascades the policy to member accounts of that OU. This means that any SCP you attach at the root of a hierarchy also applies to everything below it. An explicit deny overrules an explicit allow. An explicit allow overrules an implicit deny. By default, an SCP named *FullAWSAccess* is attached to every organization root, OU, and account. This default SCP allows all actions and all services.

i Solutions Architect Professional Exam Notes:

Remember that an SCP can only define what actions are available in an account. It does not delegate the actual permissions unlike IAM Policies. If you need to do something in your environment, you need to have the necessary policies attached first. In terms of permission hierarchy, the *deny* rule always takes precedence. This means that even if you have an IAM Policy that allows you to perform an action, if the SCP attached to that account implicitly or explicitly denies this action then you cannot perform it. Same goes with having an allow in the SCP but being implicitly or explicitly denied in the IAM Policy.

There are two common approaches to SCPs: *whitelisting* and *blacklisting*.

- **Blacklisting** applies the *FullAWSAccess* SCP, which doesn't filter out any AWS service APIs, then filters out specific APIs by blacklisting them in subsequent SCPs attached to OUs at various points in your organization's structure.
- **Whitelisting** is about modifying your SCPs to be more restrictive in allow permissions. All other actions are therefore implicitly denied. Users and roles in the affected accounts can then exercise only that level of access, even if their IAM policies allow all actions.

Shared Directory Services

If you are using AWS Managed Microsoft AD Directory, you can share this directory to other VPCs and AWS accounts within the same Region. This makes it convenient to manage different directory-aware services such as EC2 instances or local Windows servers across different VPCs and accounts. To share your directory, you first need to configure the network between the VPCs that will be communicating. You have multiple options on how to do this, such as VPC Peering, Direct Connect, Transit Gateway, VPN and so on. Once you have configured your route tables and security groups, you have two ways to share your directory:

- 1) If you are in an AWS Organization, you only need to select the accounts that you want to share the directory to. Your AWS Organization must have all features enabled and the directory must be in the organization's master account for this to work.
- 2) If you are sharing the directory to an external account, you need to initiate a handshake request and the recipient needs to accept your request.



If you have an external AD that you want to use as an authentication method for your AWS account (which is common in a hybrid environment), you can do so by using **SAML Federation**. Federation is the practice of establishing trust between a system acting as an identity provider and other systems, often called service providers, that accept authentication tokens from that identity provider. You have options on how to implement federation in AWS:

- 1) You may use AWS IAM Identity Center which works with your identity provider to handle access for your federated users and roles.
- 2) You may use IAM identity providers instead of creating IAM users in your AWS account. IAM supports providers that are compatible with OpenID Connect (OIDC) or SAML 2.0. OIDC calls the *AssumeRoleWithWebIdentity* API to trade the authentication token you get from those IdPs for AWS temporary security credentials. SAML calls AWS STS *AssumeRoleWithSAML* API, passing the ARN of the SAML provider, the ARN of the role to assume, and the SAML assertion from IdP.
- 3) You may use third-party SAML solution providers and manually configure a solution to work with AWS federation.

References:

<https://docs.aws.amazon.com/pdfs/whitepapers/latest/organizing-your-aws-environment/organizing-your-aws-environment.pdf#organizing-your-aws-environment>
https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorial_cross-account-with-roles.html
https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_type-auth.html
https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_directory_sharing.html



Using S3 Requester Pays and Bucket Policies

If you have partner accounts that need to access your S3 objects, you can have them shoulder the costs of these requests via S3 Requester Pays. You can enable this feature by simply going to your bucket properties and turning on Requester Pays. Once enabled, you must authenticate all requests involving Requester Pays buckets. Also, after you configure a bucket to be a Requester Pays bucket, requesters must include x-amz-request-payer in their requests either in the header, for POST, GET and HEAD requests, or as a parameter in a REST request.

To provide cross-account access to objects that are in your S3 buckets, configure the bucket policy to allow API access for the other account. See example policy below:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::OtherAccount:user/OtherAccountUserName"
      },
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": [
        "arn:aws:s3:::YourBucketName/*"
      ]
    }
  ]
}
```

References:

<https://docs.aws.amazon.com/AmazonS3/latest/dev/RequesterPaysBuckets.html>

<https://aws.amazon.com/premiumsupport/knowledge-center/cross-account-access-s3/>



Multi-Account Infrastructure Management

Managing multiple accounts requires you to monitor what resources need to be launched in each account and how they should be launched. You will have to employ different administration and devops techniques to make sure that users are launching resources that they only require, and that these resources comply with the organization's compliance requirements. You can perform proactive monitoring using AWS Config deployed in each account via CloudFormation script, but a better solution would be to define the accounts' infrastructures right from the get go. The key services to use for this purpose are AWS Organizations, AWS CloudFormation Stack Sets, and AWS Service Catalog.

CloudFormation Stack Sets

AWS CloudFormation StackSets is similar to your usual CloudFormation stack operation but it extends this by enabling you to create, update, or delete stacks across multiple accounts and regions with a single operation. Using an administrator account, you define and manage an AWS CloudFormation template, and use the template as the basis for provisioning stacks into selected target accounts across specified regions. As a result, you can deploy standardized infrastructures onto multiple accounts without having to log in to each of these accounts. Through the standard template, you can specify how resources will be configured at launch (such as enforcing tags) and restrict users to only use resources launched by the CloudFormation stack (via IAM policy).

Before you start using CloudFormation Stack Sets, you must first configure a trust relationship between your administrator account and the accounts you will be deploying the CloudFormation stack to. There are two methods for this:

- 1) If the target accounts are not part of your AWS Organization, you need to establish a trust relationship between the administrator and target accounts by creating IAM roles in each account.
 - a) In each target account, create a service role named **AWSCloudFormationStackSetExecutionRole** that trusts the administrator account.
 - b) Grant the service role the required permissions to perform the operations that are specified in your AWS CloudFormation template.
- 2) If the target accounts are part of your AWS Organization, you just need to enable trusted access. Note that this requires your AWS Organization to have all features enabled.
 - a) After trusted access is enabled, StackSets creates the necessary IAM roles in the administrator (AWS Organizations master) account and target accounts when you create stack sets with service-managed permissions.

Your stack set can be deployed to your entire organization or specific OUs. If your stack set targets your organization, it also targets all accounts in all OUs in the organization. If your stack set targets specified OUs, it also targets all accounts in those OUs and all child OUs. StackSets does not deploy stack instances to the



organization master account, even if the master account is in your organization or in an OU in your organization.

Service Catalog

AWS Service Catalog lets you create a portfolio of services that are approved for use in your AWS account. It helps you centrally manage commonly deployed services, achieve consistent governance, and meet compliance requirements. Users can view the approved services and select the configuration that they wish to use for deployment. In short, *what you see is what you get*. Before your end users can use your products, you must grant them permissions that allow them to access the AWS Service Catalog console, launch products, and manage launched products as provisioned products.

Catalog administrators can define these products through a CloudFormation template, which is convenient if you have a full stack of services. They can add constraints and resource tags to be used at provisioning, and then grant access to the portfolio through IAM users and groups. Updating a product in a portfolio is also very easy and can be done by updating the submitted template. You can also employ versioning to a product. When you create a new version of a product, the update is automatically distributed to all users who have access to the product, allowing the user to select which version of the product to use.

To make your AWS Service Catalog products available to users who are not in your AWS account, such as users who belong to other organizations or to other AWS accounts in your organization, you share your portfolios with them. This can be done in several ways, including **account-to-account sharing**, **organizational sharing via AWS Organizations**, and **deploying catalogs using CloudFormation stack sets**. When you share a portfolio using account-to-account sharing or organizational sharing, you allow the Service Catalog administrator of the target AWS account to import your portfolio into his or her account and distribute the products to end users in that account. The products and constraints in the imported portfolio stay in sync with changes that you make to your shared portfolio. The products in the portfolio cannot be modified by recipients, but only designate who can access the products.

- For account-to-account sharing, if a Service Catalog administrator from another AWS account shares a portfolio with you, you can import that portfolio into your account by getting its URL from the administrator.
- In AWS Organizations, you can share portfolios to an organization account, a single OU or to the whole organization which is every account in that organization.
- You can use AWS CloudFormation StackSets to launch AWS Service Catalog products across multiple regions and accounts. You can specify the order in which products deploy sequentially within regions. Across accounts, products are deployed in parallel.



i Solutions Architect Professional Exam Notes:

Should I Use CloudFormation Stack Sets? Service Catalog? Or Both?

Generally, one can see the similarities between these options. The end goal is to make sure that your users only launch what they need to and use the configuration that complies with your organization. However, there are a few pros and cons that come in between.

Stack Sets is a great option if you already have a CloudFormation template ready to deploy to multiple accounts and multiple regions. They make sure that the resulting infrastructure is similar in each location. The issue, however, is you need to have a great understanding of what each team needs for their infrastructure. It is difficult to standardize something if users have different objectives. Another issue is compliance monitoring. The resources in a CloudFormation stack can be modified after provisioning, as long as the user has the permissions to do so. This might be unacceptable for some companies.

Service Catalog is a great tool if you have specific restrictions for some of the basic services, such as EC2 or RDS. It also makes sure that end users can only deploy these services the way you designed them, so they won't need to worry about code. One issue is that products created in Service Catalog are regional. They are only visible and usable in the region you deployed them in. This can be difficult to manage when you have multiple accounts using multiple regions. Some administrators might consider creating a CI/CD pipeline, but this is additional overhead. Service Catalog also reduces the flexibility for your users with their work.

A nifty solution to cover each other's weaknesses is to combine Stack Sets with Service Catalog. That way, you can govern the infrastructure of multiple accounts and multiple regions. Users can also customize the resulting infrastructure using desired parameters. The only downside to this method is that your users need to be familiar with how products are configured.

References:

<https://aws.amazon.com/blogs/aws/use-cloudformation-stacksets-to-provision-resources-across-multiple-aws-accounts-and-regions/>
<https://aws.amazon.com/blogs/mt/managing-aws-organizations-accounts-using-aws-config-and-aws-cloudformation-stacksets/>
<https://aws.amazon.com/blogs/mt/simplify-sharing-your-aws-service-catalog-portfolios-in-an-aws-organizations-setup/>
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/what-is-cfnstacksets.html>
<https://aws.amazon.com/servicecatalog/>



Multi-Account Network Configuration

Managing networks is a huge part of your work as an AWS Solutions Architect. As an organization grows, the network requirements become more complex and intertwined. This requires careful modelling so that traffic flow won't get disrupted. It is also common for users to have more than one VPC in their environment. Oftentimes, these VPCs are placed in different locations or regions, requiring you to figure out the most appropriate solution to make sure they can communicate with each other. Some customers may also opt for a hybrid environment, wherein their AWS VPCs need to be connected to their own corporate network. AWS has a ton of options provided for these scenarios which we will discuss in the sections below.

Multi-VPC Connection and Routing

The simplest way to connect two VPCs together is to use **VPC Peering**. VPC Peering allows bidirectional communication, so once you have peered two VPCs and modified their route tables (and network ACLs if you are security-conscious) then you are good to go. VPC Peering allows you to peer VPCs in the same region, in different regions, or in different accounts. There are a few things to consider though when using VPC Peering:

- 1) The connection is not transitive. This means that you cannot peer VPC A and VPC B, where VPC B is peered with VPC C, and expect VPC A to communicate with VPC C through this connection. If you have this requirement, either peer VPC A and VPC C directly or use another solution.
- 2) When you are peered to a VPC, remember that your route table uses *longest prefix matching* to know which destination to send traffic to.
- 3) VPC Peering is **not** a great option for a hub-and-spoke model.

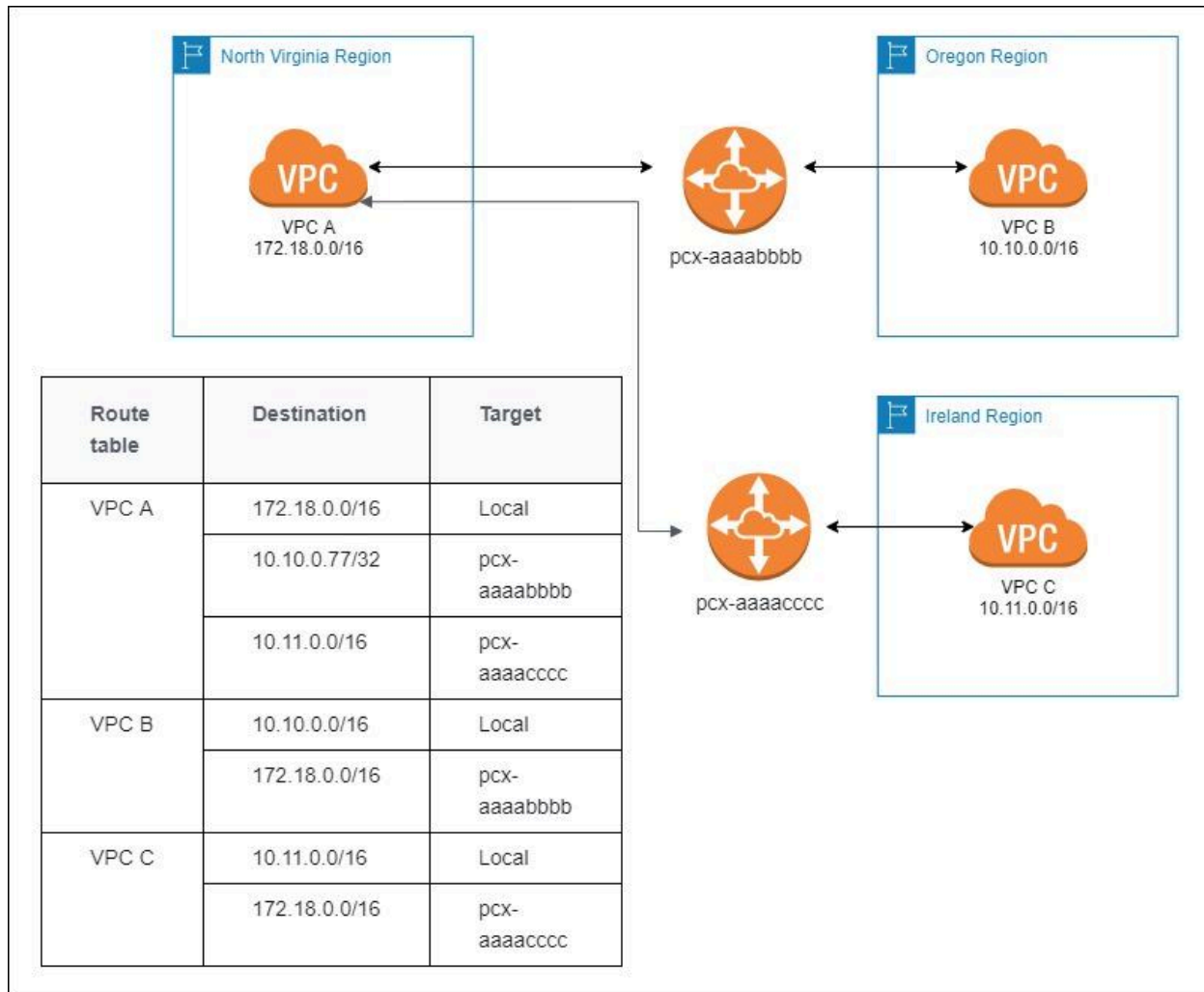


Figure: VPC Peering Between Three VPCs in Different Regions

Transit VPC allows you to build a hub-and-spoke model by using a hub VPC to connect to multiple spoke VPCs through a VPN connection leveraging BGP over IPsec. This solution can also handle transitive routing thanks to the VPN overlay. Take note that VPCs using this solution will need a VPN Gateway.

A fairly new solution that lets you build a hub-and-spoke model without a custom VPN appliance is to use **AWS Transit Gateway**. Transit Gateways enable you to connect multiple VPCs together, and beyond this, you can also link VPN solutions and AWS Direct Connect connections to a transit gateway. Transit gateways are regional services, which means you can peer transit gateways if you have VPCs in other regions, but you cannot peer transit gateways belonging to the same region. Unlike VPC Peering, you cannot reference the security group of a VPC in another VPC's security group. You have to use IP addresses or IP ranges for your rules.

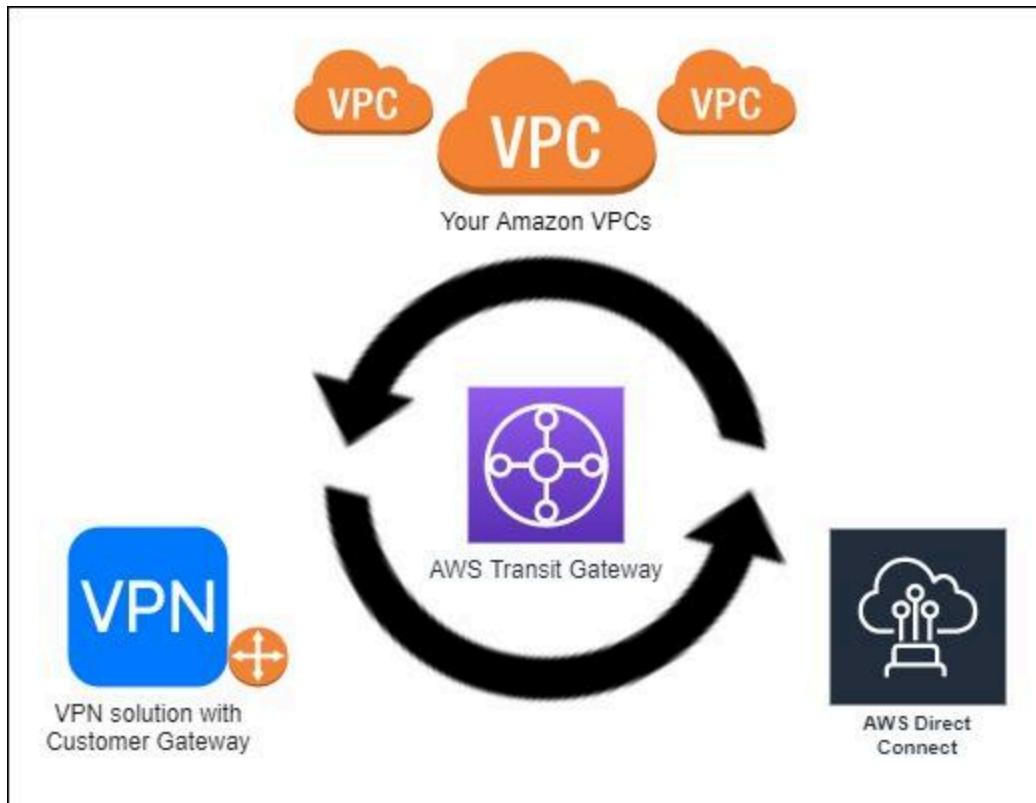


Figure: AWS Transit Gateway as a Hub-and-Spoke Solution

In case you already own an AWS Direct Connect connection on your on-premises center, you can **divide the physical connection into multiple logical connections**, one for each VPC. You can then use these logical connections for routing traffic between VPCs in the same region. You can also connect AWS Direct Connect locations in other regions using your existing WAN providers and leverage AWS Direct Connect to route traffic between regions over your WAN backbone network.

Setting up Network for Hybrid Environment

A very simple way to connect your on-premises network to AWS is through the use of a VPN. All you need is a VPN appliance in your on-premises location or use the managed AWS VPN service and a gateway in AWS to which the VPN will connect to. More often than not, customers will leverage an AWS Direct Connect line along with an IPsec VPN to ensure that traffic does not pass through the public network and they have a dedicated line for their network demands. Nonetheless, there are other solutions that you can apply with a VPN without having to purchase a pricey Direct Connect, at the expense of slower transfer speeds of course.



1. **AWS Transit Gateway supports IPsec termination for site-to-site VPN.** Register the VPCs that you need connection to in Transit Gateway and connect your VPN on the other end of the Transit Gateway service. Transit Gateway supports both Static and BGP-based Dynamic VPN connections.
2. **You can launch an EC2 instance and terminate the VPN on the instance** if you have a vendor that you or your company uses. This is a good alternative if you have compliance or compatibility issues. Afterwards, you can also peer your VPCs together to ensure that your VPN can reach your VPCs.
3. If you need one-to-one connectivity to your VPCs, **you can provision a Virtual Private Gateway (VGW).** This is a good option if you only have a few VPCs to manage. As your environment grows larger, this might not be feasible anymore so you should instead adapt AWS Transit Gateway.

If you need a dedicated line for your traffic, provision an AWS Direct Connect from a provider and have it linked to your network. AWS Direct Connect provides many benefits compared to a VPN solution, such as a private connection to AWS, lower latency, and a higher network bandwidth. There are different ways to leverage Direct Connect:

1. If you need access to resources located inside a VPC, **create a private virtual interface (VIF) to a VGW attached to the VPC.** You can create 50 VIFs per Direct Connect connection, enabling you to connect to a maximum of 50 VPCs. Connectivity in this setup restricts you to the AWS Region that the Direct Connect location is homed to. This is not the best solution if you need to connect to a bunch of VPCs.
2. If your VPCs are located in different AWS Regions, **create a private VIF to a Direct Connect gateway associated with multiple VGWs,** where each VGW is attached to a VPC. You can attach multiple private virtual interfaces to your Direct Connect gateway from connections at any Direct Connect location. You have one BGP peering per Direct Connect Gateway per Direct Connect connection. This solution will not work if you need VPC-to-VPC connectivity.
3. You can associate a Transit Gateway to a Direct Connect gateway over a dedicated or hosted Direct Connect connection running at 1 Gbps or more. To do so, you need to create a **transit VIF to a Direct Connect gateway associated with Transit Gateway.** You can connect up to 3 transit gateways across different AWS Regions and AWS accounts over one VIF and BGP peering. This is the most scalable and manageable option if you have to connect to multiple VPCs in multiple locations.
4. If you need access to AWS public endpoints or services reachable from a public IP address (such as public EC2 instances, Amazon S3, and Amazon DynamoDB), **create a VPN connection to Transit Gateway over Direct Connect public VIF.** You can connect to any public AWS service and AWS Public IP in any AWS Region. When you create a VPN attachment on a Transit Gateway, you get two public IP addresses for VPN termination at the AWS end. These public IPs are reachable over the public VIF. You can create as many VPN connections to as many Transit Gateways as you want over public VIF. When you create a BGP peering over the public VIF, AWS advertises the entire AWS public IP range to your router.



AWS Direct Connect supports both IPv4 and IPv6 on public and private VIFs. You will be able to add an IPv6 peering session to an existing VIF with IPv4 peering session (or vice versa). You can also create 2 separate VIFs – one for IPv4 and another one for IPv6.

References:

<https://d1.awsstatic.com/whitepapers/building-a-scalable-and-secure-multi-vpc-aws-network-infrastructure.pdf>

<https://d1.awsstatic.com/whitepapers/aws-amazon-vpc-connectivity-options.pdf>

<https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html>

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways.html>

Configuring DNS Resolution for your Servers

In large organizations, it is common to use an Active Directory to manage all your users, computers, and policies. Having an Active Directory also allows you to separate these identities into different groups and domains. You can then apply Group Policy Objects or GPOs to properly secure and manage your environment to meet requirements. Most of the time, computers within a domain only talk to each other within a private network. This requires having a single source of truth as to which computer owns which IP address. Therefore, you must also configure your Active Directory domain controllers to become DNS providers for your domain members. Once the DNS is up, you go into each of your domain computers and configure the network to use your domain controllers as your DNS servers. However, when you are managing multiple computers at the same time, it can be difficult to keep track if they are properly communicating with your DNS servers, especially if you're in AWS handling multiple accounts and multiple environments. To simplify this task, you can instead create DHCP option sets for your VPCs.

The Dynamic Host Configuration Protocol (DHCP) provides a standard for passing configuration information to hosts on a TCP/IP network. When you launch private instances in a non-default VPC, these instances receive an unresolvable hostname from AWS. You can assign your own domain name to your instances, and use up to four of your own DNS servers. To do so, you must specify a set of DHCP options to use with the VPC. The DHCP options set will help resolve your desired domain name to your DNS servers, which reduces the chances of misconfiguration. You can also specify your NTP servers and NetBIOS name servers if you are using any. This information, once attached to the selected VPC, are made available to all EC2 instances running in that VPC.

After you create a set of DHCP options, you can't modify them. If you want your VPC to use a different set of DHCP options, you must create a new set and associate them with your VPC. You can also set up your VPC to use no DHCP options at all. You can have multiple sets of DHCP options, but you can associate only one set of DHCP options with a VPC at a time. By default, when you create a new VPC, AWS automatically creates a set of DHCP options and associates them with the VPC. This set includes two options:



domain-name-servers=*AmazonProvidedDNS*, and domain-name=*domain-name-for-your-region*. **AmazonProvidedDNS** is an Amazon Route 53 Resolver server, and this option enables DNS for instances that need to communicate **over the VPC's Internet gateway**.

Create DHCP options set Info

Dynamic Host Configuration Protocol (DHCP) provides a standard for passing configuration information to hosts on a TCP/IP network. The options field of a DHCP message contains configuration parameters.

Tag settings

DHCP options set name - *optional*

my-dhcp-options-set-01 Give your options set a good name

DHCP options

Specify at least one configuration parameter.

Domain name Info

example.com Enter your AD domain here, like AD@company

Domain name servers Info

172.16.16.16, 10.10.10.10 Enter the IP addresses of your domain controllers

Enter up to four IP addresses, separated by commas.

NTP servers

198.51.100.2, 198.51.100.4 Enter the IP addresses of your NTP servers if you have any

Enter up to four IP addresses, separated by commas.

NetBIOS name servers

192.168.0.4, 198.168.0.5 Enter your NetBIOS IP addresses in case DNS becomes unavailable

Enter up to four IP addresses, separated by commas.

NetBIOS node type

Choose a node type ▼

We recommend that you select point-to-point (2 - P-node). Broadcast and multicast are not currently supported.

► AWS Command Line Interface command

Your VPC has attributes that determine whether instances launched in the VPC receive public DNS hostnames that correspond to their public IP addresses, and whether DNS resolution through the Amazon DNS server is



supported for the VPC. These attributes are **enableDnsHostnames** and **enableDnsSupport**. Values for these two attributes are true and false. By default, both of these are true.

VPC Rules:

- If both attributes are set to true:
 - Instances with a public IP address receive corresponding public DNS hostnames.
 - The Amazon Route 53 Resolver server can resolve Amazon-provided private DNS hostnames.
- If either or both of the attributes is set to false:
 - Instances with a public IP address do not receive corresponding public DNS hostnames.
 - The Amazon Route 53 Resolver cannot resolve Amazon-provided private DNS hostnames.
- Instances receive custom private DNS hostnames if there is a custom domain name in the DHCP options set. If you are not using the Amazon Route 53 Resolver server, your custom domain name servers must resolve the hostname.
- If you want to access the resources in your VPC using custom DNS domain names, you can create a private hosted zone in Route 53. Using custom DNS domain names defined in a private hosted zone in Route 53, or using private DNS with interface VPC endpoints (AWS PrivateLink) requires you to set both VPC attributes to true.
- The Amazon Route 53 Resolver can resolve private DNS hostnames to private IPv4 addresses for all address spaces.

Solutions Architect Professional Exam Notes:

How do I use both Active Directory and VPC Resolver for private DNS resolution in my VPC?

Once you have set up your DHCP options set with your own DNS servers, all your instances that are joined to the domain will rely on these servers for DNS-related functions. For local VPC domain names that need to be resolved to their respective IP addresses, you can configure your Active Directory to forward these types of queries to the Route 53 resolver instead via an inbound endpoint. The Route 53 resolver will then return the private IP address of those instances for you.

References:

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-dns.html>

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_DHCP_Options.html

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>